Branching-Strategien mit Git

TL;DR
Das Erstellen und Mergen von Branches ist in Git besonders einfach. Dies kann man sich in der Entwicklung zunutze machen und die einzelnen Features unabhängig voneinander in eigenen Hilfs-Branches ausarbeiten.
Es haben sich zwei grundlegende Modelle etabliert: “Git-Flow” und “GitHub Flow”.
In **Git-Flow** gibt es ein umfangreiches Konzept mit verschiedenen Branches für feste Aufgaben, welches sich besonders gut für Entwicklungmodelle mit festen Releases eignet. Es gibt zwei langlaufende Branches: `master` enthält den stabilen veröffentlichten Stand, in `develop` werden die Ergebnisse der Entwicklung gesammelt. Features werden in kleinen Feature-Branches entwickelt, die von `develop` abzweigen und dort wieder hineinmünden. Für Releases wird von `develop` ein eigener Release-Branch angelegt und nach Finalisierung in den `master` und in `develop` gemergt. Fixes werden vom `master` abgezweigt, und wieder in den `master` und auch nach `develop` integriert. Dadurch stehen auf dem `master` immer die stabilen Release-Stände zur Verfügung, und im `develop` sammeln sich die Entwicklungsergebnisse.
Der **GitHub Flow** basiert auf einem deutlich schlankeren Konzept und passt gut für die kontinuierliche Entwicklung ohne echte Releases. Hier hat man auch wieder einen `master` als langlaufenden Branch, der die stabilen Release-Stände enthält. Vom `master` zweigen direkt die kleinen Feature-Branches ab und werden auch wieder direkt in den `master` integriert.

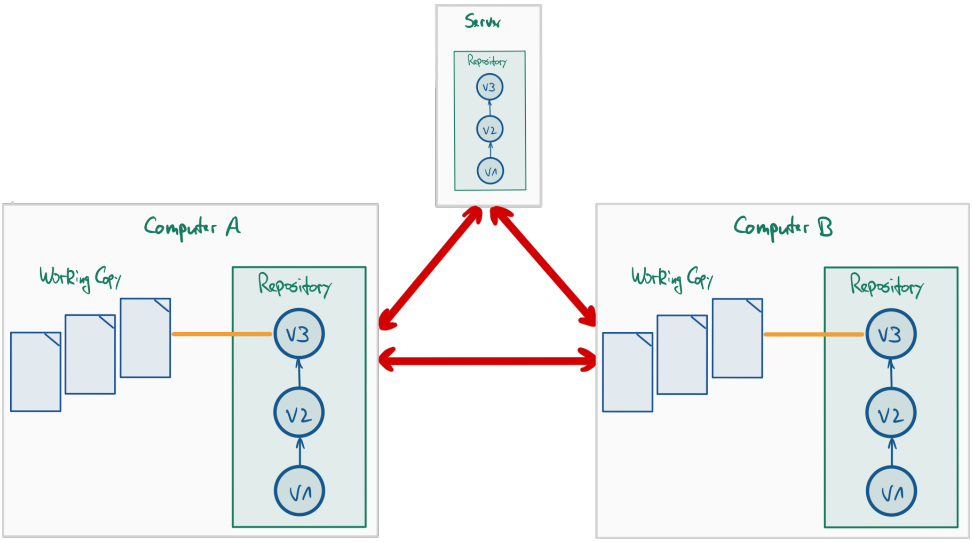
Videos

- VL Git Branching-Strategien

Unterlagen

- PDF-Version

Nutzung von Git in Projekten: Verteiltes Git (und Workflows)



Git ermöglicht ein einfaches und schnelles Branchen. Dies kann man mit entsprechenden Branching-Strategien sinnvoll für die SW-Entwicklung einsetzen.

Im Folgenden sollen also die Frage betrachtet werden: **Wie setze ich Branches sinnvoll ein?**

Umgang mit Branches: Themen-Branches

```
                I---J---K  wuppieV1
               /
          D---F  wuppie
         /
A---B---C---E  master
             \
              G---H  test
```

Branchen ist in Git sehr einfach und schnell. Deshalb wird (gerade auch im Vergleich mit SVN) gern und viel gebrancht.

Ein häufiges anzutreffendes Modell ist dabei die Nutzung von **Themen-Branches**: Man hat einen Hauptzweig (`master`). Wann immer eine neue Idee oder ein Baustein unabhängig entwickelt werden soll/kann, wird ein entsprechender Themen-Branch aufgemacht. Dabei handelt es sich normalerweise um **kleine Einheiten**!

Themenbranches haben in der Regel eine **kurze Lebensdauer**: Wenn die Entwicklung abgeschlossen ist, wird die Idee bzw. der Baustein in den Hauptzweig integriert und der Themenbranch gelöscht.

- Vorteil: Die Entwicklung im Themenbranch ist in sich gekapselt und stört nicht die Entwicklung in anderen Branches (und diese stören umgekehrt nicht die Entwicklung im Themenbranch).
- Nachteil:
  - Mangelnder Überblick durch viele Branches
  - Ursprung der Themenbranches muss überlegt gewählt werden, d.h. alle dort benötigten Features müssen zu dem Zeitpunkt im Hauptzweig vorhanden sein

Umgang mit Branches: Langlaufende Branches

```
A---B---D  master
     \
      C---E---I  develop
           \
            F---G---H  topic
```

Häufig findet man in (größeren) Projekten Branches, die über die gesamte Lebensdauer des Projekts existieren, sogenannte “langlaufende Branches”.

Normalerweise gibt es einen Branch, in dem stets der stabile Stand des Projekts enthalten ist. Dies ist häufig der `master`. In diesem Branch gibt es nur sehr wenige Commits: normalerweise nur Merges aus dem `develop`-Branch (etwa bei Fertigstellung einer Release-Version) und ggf. Fehlerbehebungen.

Die aktive Entwicklung findet in einem separaten Branch statt: `develop`. Hier nutzt man zusätzlich Themen-Branches für die Entwicklung einzelner Features, die nach Fertigstellung in den `develop` gemergt werden.

Kleinere Projekte kommen meist mit den zwei langlaufenden Branches in der obigen Darstellung aus. Bei größeren Projekten finden sich häufig noch etliche weitere langlaufende Branches, beispielsweise "Proposed Updates" etc. beim Linux-Kernel.

- Vorteile:
  - Mehr Struktur im Projekt durch in ihrer Semantik wohldefinierte Branches
  - Durch weniger Commits pro Branch lässt sich die Historie leichter verfolgen (u.a. auch aus bestimmter Rollen-Perspektive: Entwickler, Manager, ...)
- Nachteile: Bestimmte "ausgezeichnete" Branches; zusätzliche Regeln zum Umgang mit diesen beachten

Komplexe Branching-Strategie: Git-Flow

```
A---B--------------------G---J1  master
     \                   / \ /
      \                 /   X  fix
       \               /     \
        C-------------F----I--J2  develop
         \           / \  /
          \         /   H1  featureB
           \       /
            D1----D2  featureA
             \
              E1---E2---E3---E4---E5  featureC
```

Das Git-Flow-Modell von Vincent Driessen (nvie.com/posts/a-successful-git-branching-model) zeigt einen in der Praxis überaus bewährten Umgang mit Branches. Lesen Sie an der angegebenen Stelle nach, besser kann man die Nutzung dieses eleganten Modells eigentlich nicht erklären :-)

Git-Flow: Hauptzweige *master* und *develop*

```
A---B-------E---------------J  master
     \     /               /
      C---D---F---G---H---I---K  develop
```

Bei Git-Flow gibt es zwei langlaufende Branches: Den `master`, der immer den stabilen Stand enthält und in den *nie* ein direkter Commit gemacht wird, sowie den `develop`, wo letztlich (ggf. über Themenbranches) die eigentliche Entwicklung stattfindet.

Änderungen werden zunächst im `develop` erstellt und getestet. Wenn die Features stabil sind, erfolgt ein Merge von `develop` in den `master`. Hier kann noch der Umweg über einen `release`-Branch genommen werden: Als "Feature-Freeze" wird vom `develop` ein `release`-Branch abgezweigt. Darin wird das Release dann aufpoliert, d.h. es erfolgen nur noch kleinere Korrekturen und Änderungen, aber keine echte Entwicklungsarbeit mehr. Nach Fertigstellung wird der `release` dann sowohl in den `master` als auch `develop` gemergt.

Git-Flow: Weitere Branches als Themen-Branches

```
A---B--------------------I-------------K  master
     \                  /             /
      C------------F----H-------------J---L  develop
       \          / \  /             /
        \        /   G1  featureB   /
         \      /                  /
          D1---D2  featureA       /
           \                     /
            E1---E2---E3---E4---E5  featureC
```

Für die Entwicklung eigenständiger Features bietet es sich auch im Git-Flow an, vom `develop` entsprechende Themenbranches abzuzweigen und darin jeweils isoliert die Features zu entwickeln. Wenn diese Arbeiten eine gewisse Reife haben, werden die Featurebranches in den `develop` integriert.

Git-Flow: Merging-Detail

```
---C--------E  develop
    \      /                git merge --no-ff
     D1---D2  featureA
```

vs.

```
---C---D1---D2  develop      git merge
```

Wenn beim Mergen ein "*fast forward*" möglich ist, würde Git beim Mergen eines (Feature-) Branches in den `develop` (oder allgemein in einen anderen Branch) *keinen* separaten Commit erzeugen (Situation rechts in der Abbildung).

Damit erscheint der `develop`-Branch wie eine lineare Folge von Commits. In manchen Projekten wird dies bevorzugt, weil die Historie sehr übersichtlich aussieht.

Allerdings verliert man die Information, dass hier ein Feature entwickelt wurde und wann es in den `develop` integriert wurde (linke Seite in obiger Abbildung). Häufig wird deshalb ein extra Merge-Commit mit `git merge --no-ff <branch>` (extra Schalter "`--no-ff`") erzwungen, obwohl ein "*fast forward*" möglich wäre.

Anmerkung: Man kann natürlich auch über Konventionen in den Commit-Kommentaren eine gewisse Übersichtlichkeit erzwingen. Beispielsweise könnte man vereinbaren, dass alle Commit-Kommentare zu einem Feature "A" mit "`feature a:`" starten müssen.

Git-Flow: Umgang mit Fehlerbehebung

```
A---B---D--------F1  master
     \   \      /
      \   E1---E2  fix
       \        \
        C1-------F2  develop
```

Wenn im stabilen Branch (also dem `master`) ein Problem bekannt wird, darf man es nicht einfach im `master` fixen. Stattdessen wird ein extra Branch vom `master` abgezweigt, in dem der Fix entwickelt wird. Nach Fertigstellung wird dieser Branch sowohl in den `master` als auch den `develop` gemergt, damit auch im Entwicklungszweig der Fehler behoben ist.

Dadurch entspricht jeder Commit im `master` einem Release.

Vereinfachte Braching-Strategie: GitHub Flow

```
A---B---C----D-----------E  master
     \   \  /           /
      \   ta1  topicA  /
       \              /
        tb1---tb2---tb3  topicB
```

Github verfolgt eine deutlich vereinfachte Strategie: “GitHub Flow” (vgl. “GitHub Flow” (S. Chacon) bzw. “GitHub flow” (GitHub, Inc.)).

Hier ist der stabile Stand ebenfalls immer im `master`. Features werden ebenso wie im Git-Flow-Modell in eigenen Feature-Branches entwickelt.

Allerdings zweigen Feature-Branches *immer direkt* vom `master` ab und werden nach dem Test auch immer dort wieder direkt integriert (es gibt also keine weiteren langlaufenden Branches wie `develop` oder `release`).

In der obigen Abbildung ist zu sehen, dass für die Entwicklung eines Features ein entsprechender Themenbranch vom `master` abgezweigt wird. Darin erfolgt dann die Entwicklung des Features, d.h. mehrere Commits. Das Mergen des Features in den `master` erfolgt dann aber nicht lokal, sondern mit einem “Pull-Request” auf dem Server: Sobald man im Feature-Branch einen “diskussionswürdigen” Stand hat, wird ein **Pull-Request** (*PR*) über die Weboberfläche aufgemacht (streng genommen gehört dies in die Kategorie “Zusammenarbeit” bzw. “Workflows”; außerdem gehört ein PR nicht zu Git selbst, sondern zum Tooling von Github). In einem PR können andere Entwickler den Code kommentieren und ergänzen. Jeder weitere Commit auf dem Themenbranch wird ebenfalls Bestandteil des Pull-Requests. Parallel laufen ggf. automatisierte Tests etc. und durch das Akzeptieren des PR in der Weboberfläche erfolgt schließlich der Merge des Feature-Branches in den `master`.

Diskussion: Git-Flow vs. GitHub Flow

In der Praxis zeigt sich, dass das Git-Flow-Modell besonders gut geeignet ist, wenn man tatsächlich so etwas wie “Releases” hat, die zudem nicht zu häufig auftreten.

Das GitHub-Flow-Vorgehen bietet sich an, wenn man entweder keine Releases hat oder diese sehr häufig erfolgen (typisch bei agiler Vorgehensweise). Zudem vermeidet man so, dass die Feature-Branches zu lange laufen, womit normalerweise die Wahrscheinlichkeit von Merge-Konflikten stark steigt. **Achtung**: Da die Feature-Branches direkt in den `master`, also den stabilen Produktionscode gemergt werden, ist es hier besonders wichtig, *vor* dem Merge entsprechende Tests durchzuführen und den Merge erst zu machen, wenn alle Tests “grün” sind.

Hier ein paar Einstiegsseiten für die Diskussion, die teilweise sehr erbittert (und mit ideologischen Zügen) geführt wird (erinnert an die Diskussionen, welche Linux-Distribution die bessere sei):

- Git-Flow-Modell von Vincent Driessen
- Kurzer Überblick über das GitHub-Flow-Modell
- Diskussion des GitHub-Flow-Modells (Github)
- Luca Mezzalira: "Git-Flow vs Github Flow"
- Scott Schacon, Autor des Pro-Git-Buchs
- Noch eine (längere) Betrachtung (Robin Daugherty)

Wrap-Up

- Einsatz von Themenbranches für die Entwicklung
- Unterschiedliche Modelle:
  - Git-Flow: umfangreiches Konzept, gut für Entwicklung mit festen Releases
  - GitHub Flow: deutlich schlankeres Konzept, passend für kontinuierliche Entwicklung ohne echte Releases

Zum Nachlesen

- Chacon und Straub (2014, Kap. 3)
- Driessen (2010)
- Chacon (2013)
- GitHub Inc. (2022)
- Atlassian Pty Ltd (2022)
- Github Inc. (2022)

Lernziele

- k3: Einsatz von Themenbranches in der Entwicklung
- k3: Git-Flow-Modell anwenden
- k3: GitHub Flow-Modell anwenden

Quellen

Atlassian Pty Ltd. 2022. „Become a Git Guru." 2022. https://www.atlassian.com/git/tutorials.
Chacon, S. 2013. „GitHub Flow". 2013. https://githubflow.github.io/.
Chacon, S., und B. Straub. 2014. *Pro Git*. 2. Aufl. Apress. https://git-scm.com/book/en/v2.
Driessen, V. 2010. „Git-Flow: A Successful Git Branching Model". 2010. https://nvie.com/posts/a-successful-git-branching-model/.
Github Inc. 2022. „Git Cheat Sheets". 2022. https://training.github.com/.
GitHub Inc. 2022. „GitHub Flow". 2022. https://docs.github.com/en/get-started/quickstart/github-flow.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** ba2c853 (Lecture: remove 'quizzes' from the metadata to avoid any confusion with the new lecture quizzes, 2025-04-09)